

ON BUILDING

SYSTEMS

Tool Use and RAG in Prod



#ASLI ENGINEERING

BY

ARPIT BHAYANI

Agenda

1. Tool use and how tools work 10 mins
 2. Tool schema design 7 mins
 3. Parallel Tool Calls and Partial Failures 5 mins
-
4. Hybrid Search and Metadata Filtering
 5. Re-ranking : Cross Encoders and Bi-encoders
 6. Query Rewriting
 7. Semantic Caching
-

10 mins QnA

Prototypes: 7

System: RAG with 10M documents, no Hallucinations

Appendix

Evaluating RAG with RAGAS

Streaming Input with Tool Calls

external to model

Tool Use

Extends model's capabilities to call "external systems"

eg: given a city, get me temperature

given an expression, solve it

run a bash command



Provide tools with your prompt

Model will use the tool (if required) and proceed

how?

how does it pick the right one?

First thing: Model never calls a function!

↗ about the function that

Tools are defined as a list of functions + metadata

helps LLM decide

This is sent to model in "user message"

Application intercepts the response

↳ model reads the message and

Application invokes the function

decides if external call is needed

Application sends the result to model

← if yes, what arguments?

Model proceeds....

↳ tool-result message

↳ role: reasoning and argument construction ONLY

Prototype: Tool call demo

Deal with rate limits as well

↳ Register a simulated get-weather tool call

↳ fine opt prompt and see LLM "use" it

↳ Failure handling

Importance of Schema Design ^{Tool call}

JSON schema is contract b/w your intent and the model's behaviour.

* ambiguous schema is most common root cause of tool failure and is 'invisible' until production

Three rules:

1. not just description but specify the semantics

search for products

search for products by name, category, or sku.

use only when user is explicitly looking to find
or filter catalog items. Do not use for pricing
questions or stock checks - those have other dedicated tools.

2. required and optional parameters

if you do not mention, and param is optional, model may hallucinate ^x

↳ param is mandatory, → no value pass ^x

provide default values for model to use

3. Enums are cheapest reliability primitive

↳ keeps things readable & reduces hallucination

* large number of tools: model gets confused

↳ hence dynamic tool loading

what category tool?

which specific tool?

Prototype: Tool schema Calibration

model gets confused

↳ dealing with ambiguous tool descriptions

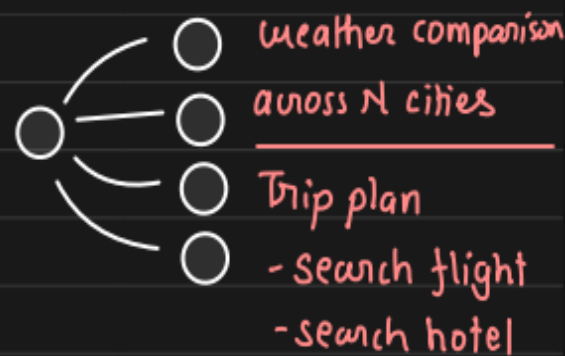
Parallel Tool Calls and Failure Handling

1. Let model emit multiple tool call requests in single turn

Orchestration layer fans out concurrently
collect result and send together

improves overall time to complete

use it when operations are independent



2. Handling Failure (it depends)

- send only successful tool call response

- abort entire execution

- prevent → absorb → fail gracefully

4 websearch calls,

lookup across multiple
knowledge bases

Answer to any question could well be IT DEPENDS

20/50 documents certainly having the right answer

improves recall



Traditional keyword based search

Hybrid Search: Dense + Sparse (BM25)

Pure semantic search works best when there is semantic similarity

How do I reset my password → Account Recovery Steps

but what about error code 1099-MISC?

BM-25 is opposite

Fix: Run both and merge and rerank!

brings reciprocal ranks together

Reciprocal Rank Fusion

k is usually 60 → $\frac{1}{1}, \frac{1}{2}, \dots$

$$RAF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

Fusion is better

← $\frac{1}{61}, \frac{1}{62}, \dots$

ES ↑ Pinerone ↑

* Weaviate and ES supports Hybrid Search Natively... otherwise scatter-gather

Metadata Filtering

apart from retrieval relevance, this is important for Enterprise RAG (RBAI)

what was our Q1 2026 revenue?

↓ applies hard constraints before or after the vector search

extract and pass filters to vector store

↓
pre-filter

↓
post-filter

faster (vector db should support metadata filter)

wastes retrieval budget

Prototype: Reciprocal Rank Fusion

Prototype: Metadata Filtering

Re-ranking: Cross Encoders

→ good enough for most

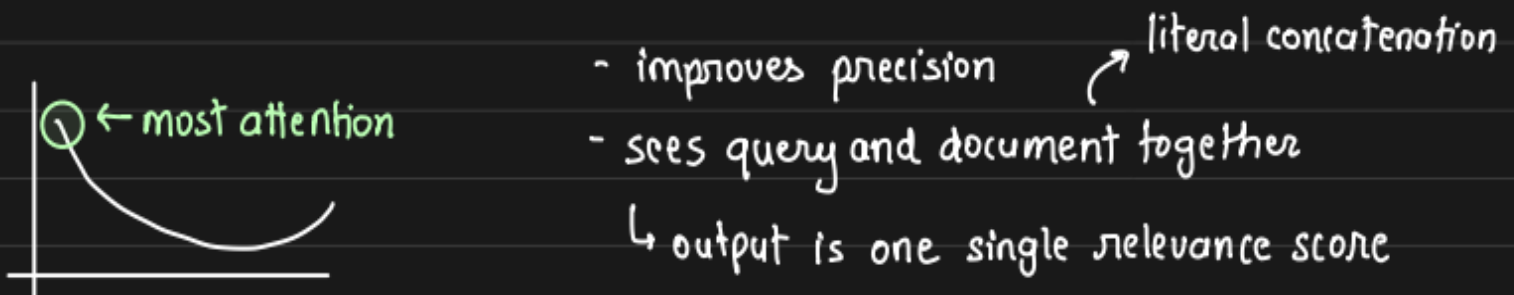
Bi-encoders: $\left. \begin{array}{l} \text{Query} \rightarrow \text{embedding} \\ \text{document} \rightarrow \text{embedding} \end{array} \right\} \text{check cosine similarity}$

Fast at query time, scalable, offline indexing → relevance = vector proximity

No cross attention i.e. query and doc never see each other
weaker on negation, exact term, etc

Takes $\langle q, d \rangle$ pair as single input and outputs a relevance score.

"Every token in query attends every token of document (across all docs)"



lost in the middle problem cross-encoder / ms-marco-MiniLM-L-6-v2

much higher accuracy - model can compare every token in q to every token in d
handles negation, exact matches, etc

Slow, does not scale (50 doc v/s million doc per query)

no indexing, ~10ms latency

Prototype: Cross-Encoders v/s Bi-encoders

1M docs, 20ms per pair
5.5 hours per query!
↓
could be done on candidate docs only

Hypothetical doc embedding

Query Rewriting with HyDE

Query Document Gap is most persistent failure mode in RAG

Start with this

Short, conversational

semantically distant from doc/corpus

why is my pipeline slow?

Throughput degradation is.....

similarity search between

$\hat{d}$ and $d \in D$

works when corpus vocab

is very diff than user query.

ex: scientific papers, legal texts, internal jargon-heavy documentation

Prototype: HyDE Query Rewrite

1. Expand or rephrase user queries

"generate 3 alternate phrasings...."

2. Hypothetical Document Embedding

"write a short document that will answer this question. Be factual. if you don't know the answer, write what the answer would typically look like"

time and money

Semantic Caching

Every call to LLM cost latency and tokens
↓
intercepts query before it reaches model
and returns a cached response

Not exact query but
semantically same

* this works best when - static data with no personalization

↳ Ex: Ask AI on your API / integration documentation

How do I integrate libx?
I want help to integrate libx } on libx website

Ex: CX support chatbot with bounded query space (60%)

Implementation: → normalize or store as is

1. incoming query
2. index it in vector database w/ response
3. during lookup, find similar queries

how? and what's
the threshold

↓
cost v/s correctness

Too low → wrong answers

Too high → hit rate collapses

Reliable way:

on your query distribution find your 'sweet spot'

Start with 0.85 cosine similarity

Prototype: Semantic Caching and Similarity Calibration

↳ identify 'duplicates' while rejecting 'traps'

Find optimal similarity threshold
maximizing #cached responses (HR)
minimizing incorrect results

queries that look similar but
have different meanings.

500 queries → embeddings → index

for different similarity thresholds (0.75 to 0.95)

↳ first in cache population

↳ calculate hit rate (# found query in cache)

↳ calculate accuracy / precision

seed, paraphrased, and near-miss queries

- Explain the ending of movie Arrival (seed)

- Please clarify the conclusion of the movie Arrival (paraphrase)

- Who directed the movie Arrival (near-miss)

seed with different
intent.

accuracy = $\frac{\text{correct hits}}{\text{total hits}}$ (paraphrase matching the seed)

incorrect hit (false positive) (near miss matching seed)

RAK with 10M docs, zero hallucinations

Answers natural language queries

correctness ↑↑

guarantee: system must not assert any claim

that cannot be traced to retrieved passage

what we are not? doc management system, ingestion pipeline, etc

10M docs, 2KB each, 200 QPS. ~1K docs updated/day

user interacting via web interface

We need not serve data from just-updated document (~15mins)

Solutioning...

✓ 1. How do we store the docs? and where?

✓ 2. What query api look like?

✓ 3. What response looks like?

✓ 4. Which databases, and why?

✓ 5. Capacity estimation: query, storage

✓ 6. Read Path and Query Pseudocode

✓ 7. What should be returned? [2]

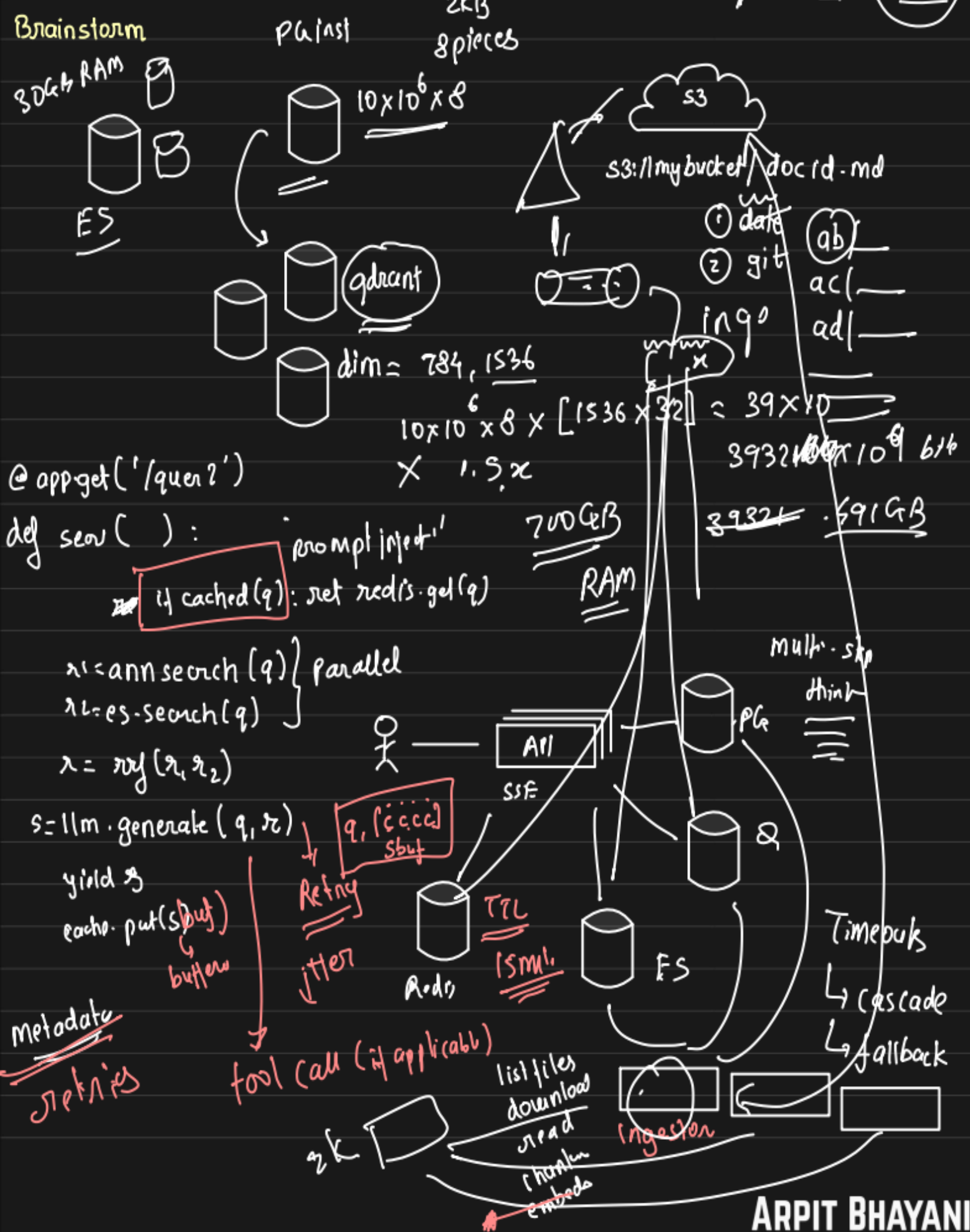
✓ 8. Failures in read path?

✓ 9. Write path

↳ S3 to databases

✓ 10. What are we missing?

deletions $\frac{1000}{24 \times 60} \times 5$ $\frac{5 \text{ doc}}{\text{min}}$ Azure AI search abstract cell
 'Query today's doc' $10 \times 10^6 \times 2 \text{ KB} = \underline{\underline{20 \text{ GB}}}$



no factual claims unsupported
by retrieved documents

for RAG, to ensure **high correctness**....

1. High quality retrieval $\rightarrow$ High precision and High recall
2. asking LLM to not hallucinate i.e. answer faithfulness

Scale: 10M docs, say 2KB each = 20GB raw data

QPS = 200 updates $\sim$ 1000 docs/day } **very read heavy**

Eventual consistency with $\sim$ 5 mins SLO $\rightarrow$ **Eventual consistency**

if irrelevant doc $\rightarrow$ **Sorry, I could not find anything relevant.**

1. query(q)
2. ingest(doc)
3. get-doc(id)
4. get-passages(doc.id)

Query latency: p50: 1.5sec
p99: 5sec
long pole: LLM inference call

} Time to 1st Token

question: _____

filters: {

date from and to,

source domains,

tag, topics, etc

}

1. 10 million doc @ top-level in bucket

2. some organization

* **Git inspiration**

answer: _____,

refused: True/false,

refusal-reason: _____,

passages: [{ docid, passage-id, text, score, highlight idx }]

confidence: _____

query-id: _____ $\leftarrow$ **Tracing**

S3://myrag/doc1.txt



10 million?

ARPIT BHAYANI

capacity estimation

$$\frac{50k \text{ DAU} \times 8 \text{ Q/day} \times 4.5 (\text{peak multiplier})}{86400} = 200 \text{ QPS}$$

So, we design our system for 250 QPS → with custom shard key
on default consistent hashing
scatter gather



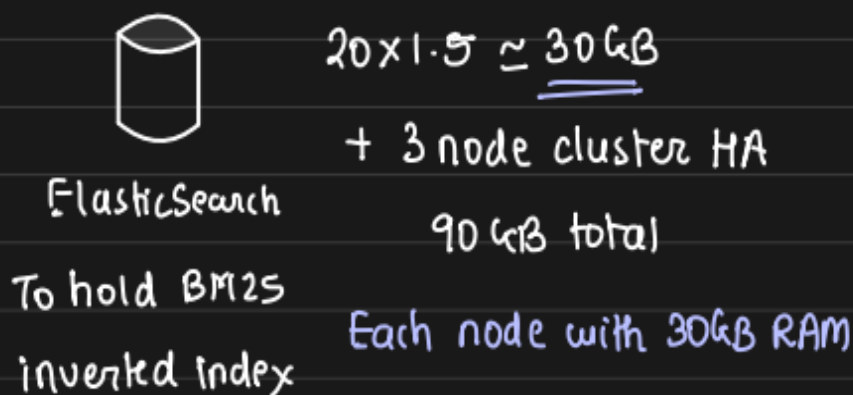
serving passage metadata

upon post RAG

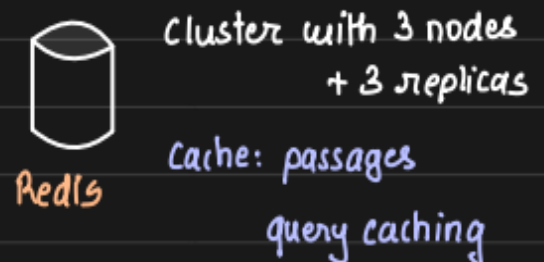
Integrity + consistency (doc - passage), + HNSW graph overhead $\sim 1.5x$ → RAM
JSONB, Replication Total vector index storage = 700GB

Hence, we need a distributed vector database

* we can apply Product Quantization to reduce $8x$ but at cost of recall.



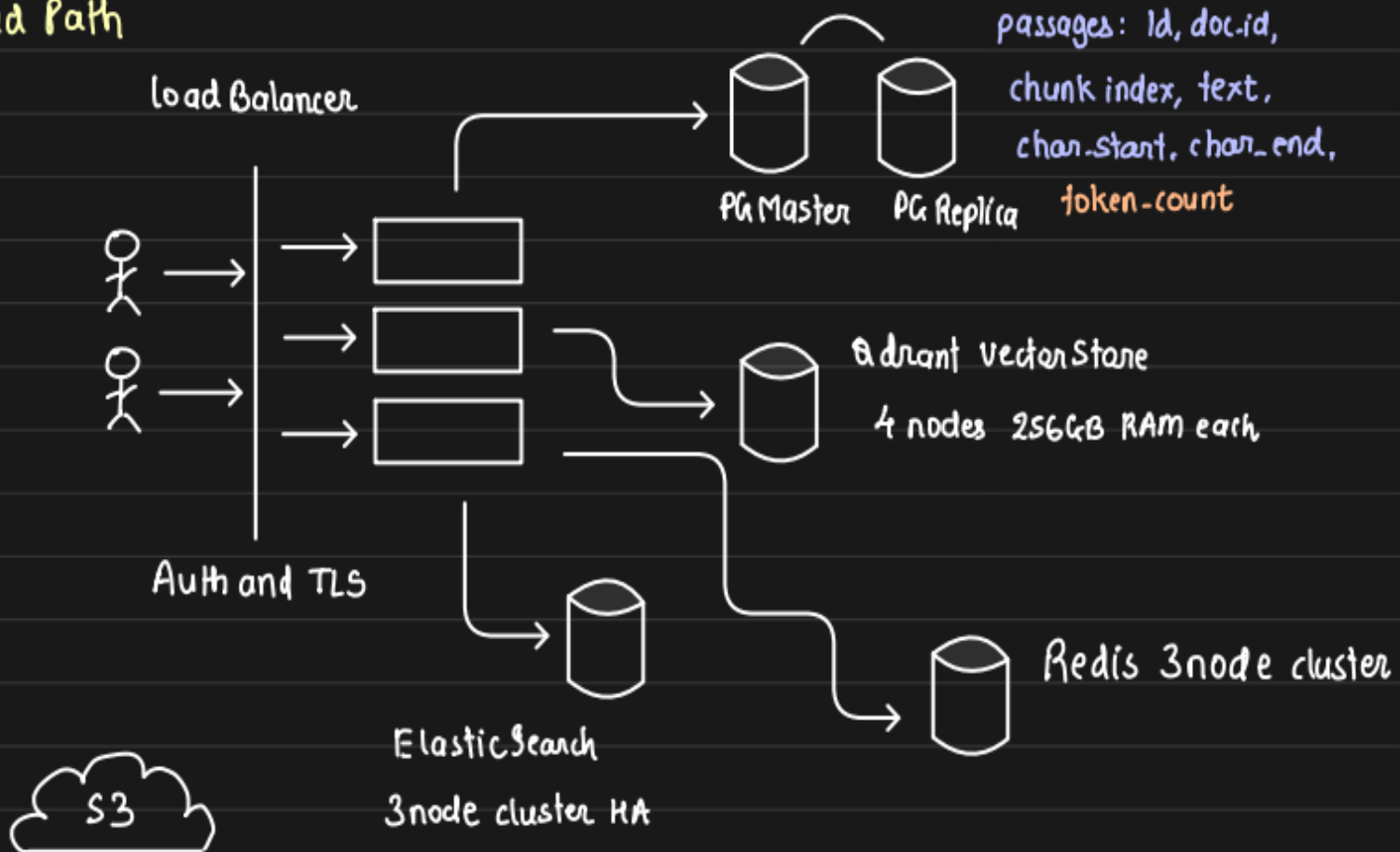
English analyzer with
stemming and stop words



* if no personalization
? data is static

docs: id, text, title, url,
date, tags, is-deleted...

Read Path



```
function respond(question: string) -> {answer: string, passages: [], refused: bool}:
```

```
    query_vector = embed(question)
```

```
    dense_results = qdrant_ann_search(query_vector, top_k=40)
```

```
    sparse_results = elasticsearch_bm25(question, top_k=40)
```

```
    candidates = rrf_fusion(dense_results, sparse_results)
```

```
    ranked = cross_encoder_rerank(question, candidates, top_k=40)
```

```
    if no passage in ranked has score > 0.4:
```

```
        return refused_response("low_retrieval_confidence")
```

```
    top_passages = ranked[:5]
```

```
    prompt = build_prompt(question, top_passages)
```

```
    raw_answer = llm_generate(prompt, temperature=0)
```

```
    if raw_answer starts with "INSUFFICIENT_CONTEXT":
```

```
        return refused_response("no_relevant_passages")
```

```
    claims = extract_atomic_claims(raw_answer)
```

```
    for each claim in claims:
```

```
        if not nli_entailed(claim, top_passages):
```

```
            return refused_response("unverified_claim")
```

```
    return {answer: raw_answer, passages: top_passages, refused: false}
```

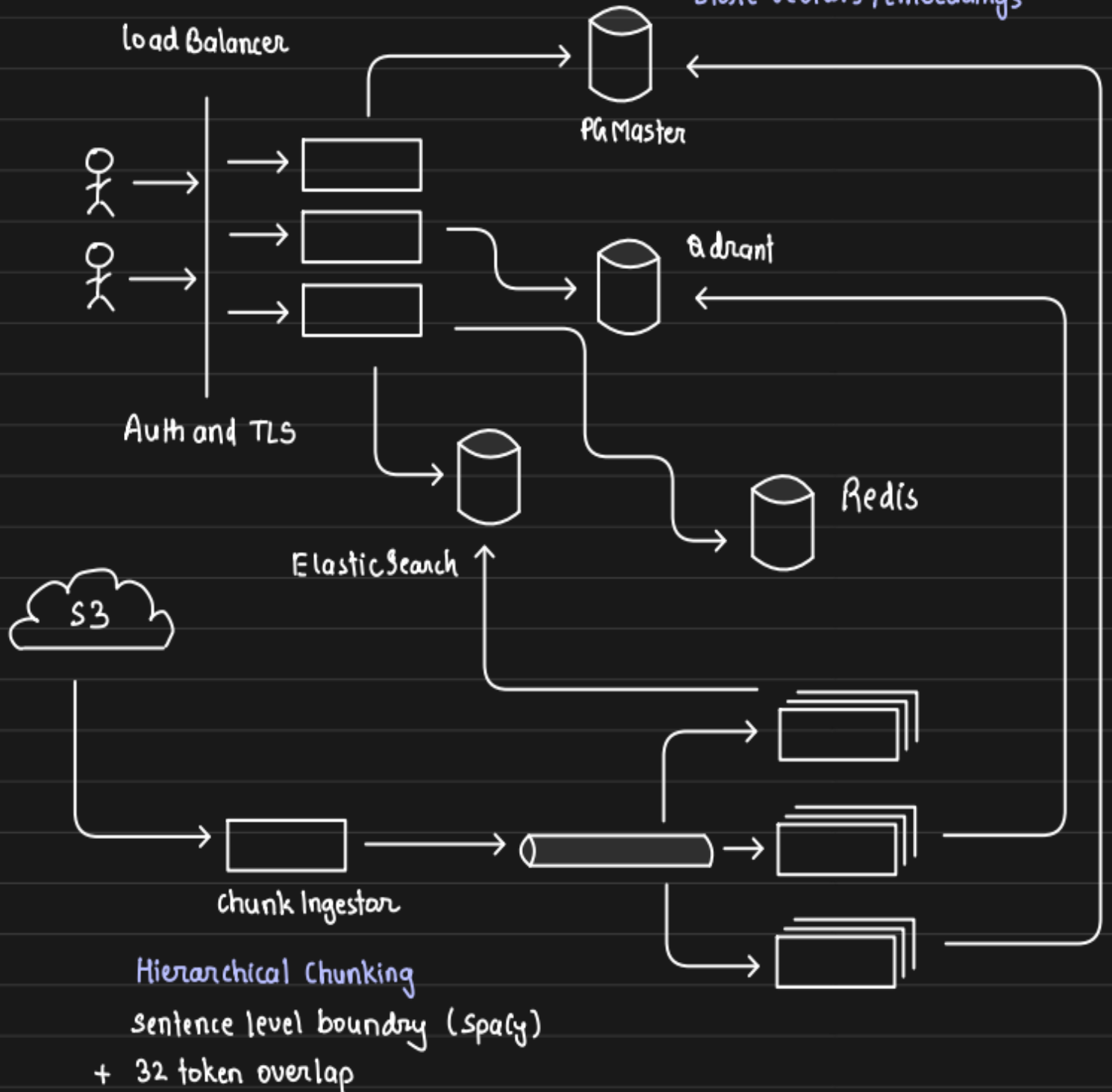
} Fact checking
system we discussed in 1st session

microsoft/hanier,
Gwen 3 (8B), BGE-M3

OpenAI or

→ OSS embedding

store passages
store vectors / embeddings



Post reads and Exercises

You can find them on course portal.

Appendix

Streaming Tool Calls

Most important metric: TTFT

bad UX

if tool call takes time, user will have to wait!

like everything else

what if tool input arrives as partial JSON fragments

↳ so, wait until the entire input is available → keep buffering

```
import anthropic

client = anthropic.Anthropic()

with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=[{
        "name": "get_weather",
        "description": "Get current weather for a location",
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {"type": "string"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["location"]
        }
    }],
    messages=[{"role": "user", "content": "What is the weather in Tokyo?"}]
) as stream:
    current_tool_input = ""
    current_tool_name = None

    for event in stream:
        if event.type == "content_block_start":
            if event.content_block.type == "tool_use":
                current_tool_name = event.content_block.name
                current_tool_input = ""
            elif event.type == "content_block_delta":
                if event.delta.type == "input_json_delta":
                    current_tool_input += event.delta.partial_json
                elif event.delta.type == "text_delta":
                    print(event.delta.text, end="", flush=True)
            elif event.type == "content_block_stop":
                if current_tool_name:
                    import json
                    parsed_input = json.loads(current_tool_input)
                    result = execute_tool(current_tool_name, parsed_input)
                    current_tool_name = None
```

"delete that record..."

↓

half baked input / tool call with
no input could be fixed.

→ Accumulate

→ input complete

→ execute

Evaluating RAG with RAGAS

Naive RAG evaluation $\rightarrow$ Qualitative "does it seem right?"

+ manual feedback loops are slow!

RAGAS: - no need for human annotated ground truth
- ensure changes (optimizations) do not downgrade quality

Context Precision:

$\nearrow$ retriever or re-ranker

- measures retriever's ability to rank relevant chunks higher
 - penalizes systems that bury relevant below 'noise'
 - use LLM to rank each chunk as RELEVANT / NOT
- for every chunk at k , measure precision @ k
- lost-in-the-middle problem



$$\begin{array}{lll} p@1 = 1 & p@3 = 2/3 & p@5 = 0 \\ p@2 = 0 & p@4 = 0 & \end{array} \quad \text{context precision} = (1 + 0.67) / 2 = 0.835$$

$\uparrow$
#relevant chunks

Faithfulness score:

$\nearrow$ measure of hallucination

measures factual consistency of generated answer

- break answer into atomic claims (using LLM)
- 1: context supports claim 0: context does not support the claim

$$\text{faithfulness} = \frac{\text{\#claims supported}}{\text{\#claims}}$$

Prototype: Regression with RAGAS